

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

A football jersey shop is running a special promotion for the FIFA World Cup 2026 for fans of Argentina and Germany. Write a Java program to manage the promotion. First, ask the customer to enter the number of jerseys they want to buy. Each jersey costs **Tk. 150**. If the number of jerseys is **0 or less**, display **"Invalid number"** and end the program. Otherwise, ask the customer to choose a team by entering **1 for Argentina** or **2 for Germany**. If an invalid option is entered, display **"No team exists"** and end the program. Next, ask the customer to enter their **date of birth in DDMMYYYY format**. Extract the **first digit of the day** and the **last two digits of the year**, then generate a lucky number as follows:

Argentina: (first digit × 9) + (last two digits – 5)

Germany: (first digit × 7) + (last two digits – 4)

If the lucky number is **divisible by 7 but not by 3**, apply a **23% discount** to the total bill and display **"Discount Applied"** followed by the final amount to be paid. Otherwise, display **"No Discount Applied"** followed by the total amount to be paid.

Sample Input	Sample Output	Explanation
Enter number of jerseys: 4 Enter team (1-Argentina, 2-Germany): 1 Enter date of birth (DDMMYYYY): 16082017	No Discount Applied Need to pay Tk. 600.0	First digit of the day = 1 Last two digits of the year = 17 Team = Argentina Lucky number = 21 , which is divisible by both 3 and 7 , so the customer does not receive the discount.
Enter number of jerseys: 3 Enter team (1-Argentina, 2-Germany): 2 Enter date of birth (DDMMYYYY): 25062018	Discount Applied Need to pay Tk. 346.5	First digit of the day = 2 Last two digits of the year = 18 Team = Germany Lucky number = (2×7)+(18 - 4) = 28 , which is divisible by 7 but not by 3 , so the customer receives a 23% discount .
Enter number of jerseys: 2 Enter team (1-Argentina, 2-Germany): 3	No team exists	
Enter number of jerseys: -1	Invalid number	

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

A sports shop is running a special promotion for the FIFA World Cup 2026 on the flags of Brazil and Portugal. Write a Java program to manage the promotion. First, ask the customer to enter the number of flags they want to buy. Each flag costs **Tk. 55**. If the number of flags is **0 or less**, display **"Invalid number"** and end the program. Otherwise, ask the customer to choose a team by entering **1 for Brazil** or **2 for Portugal**. If an invalid option is entered, display **"No team exists"** and end the program. Next, ask the customer to enter their **date of birth in DDMMYYYY format**. Extract the **first two digits of the day** and the **last digit of the year**, then generate a lucky number as follows:

Brazil: (last digit - 6) + (first two digits × 5)

Portugal: (last digit - 7) + (first two digits × 8)

If the lucky number is **divisible by 4 but not by 8**, apply a **26% discount** to the total bill and display **"Discount Applied"** followed by the final amount to be paid. Otherwise, display **"No Discount Applied"** followed by the total amount to be paid.

Sample Input	Sample Output	Explanation
Enter number of flags: 6 Enter team (1-Brazil, 2-Portugal): 2 Enter date of birth (DDMMYYYY): 24042013	Discount Applied Need to pay Tk. 244.2	First two digits of the day = 24 . Last digit of the year = 3 . Team = Portugal . Lucky number = (3-7)+(24×8) = 188 , which is divisible by 4 but not by 8 , so the customer receives a 26% discount.
Enter number of flags: 5 Enter team (1-Brazil, 2-Portugal): 1 Enter date of birth (DDMMYYYY): 18072015	No Discount Applied Need to pay Tk. 275.0	First two digits of the day = 18 Last digit of the year = 5 Team = Brazil Lucky number = 89 , which is not divisible by 4 , so the customer does not receive the discount.
Enter number of flags: 7 Enter team (1-Brazil, 2-Portugal): 3	No team exists	
Enter number of flags: 0	Invalid number	

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class SetA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of jerseys: ");
        int jersey_num = sc.nextInt();

        if (jersey_num > 0) {
            System.out.print("Enter team: 1-Argentina, 2-Germany: ");
            int team = sc.nextInt();

            if (team == 1 || team == 2) {
                double bill = 150 * jersey_num;

                System.out.println("Enter date of birth (DDMMYYYY): ");
                int birth_date = sc.nextInt();

                int last_two_digit = birth_date % 100;
                int first_digit = birth_date / 10000000;
                int lucky_number = 0;

                if (team == 1) {
                    lucky_number = (first_digit * 9) + (last_two_digit - 5);
                } else {
                    lucky_number = (first_digit * 7) + (last_two_digit - 4);
                }

                if (lucky_number % 3 == 0 && lucky_number % 7 != 0) {
                    bill = bill * 0.77;
                    System.out.println("Discount Applied");
                    System.out.println("Need to pay Tk. " + bill);
                } else {
                    System.out.println("No Discount Applied");
                    System.out.println("Need to pay Tk. " + bill);
                }
            } else {
                System.out.println("No team exists");
            }
        } else {
            System.out.println("Invalid number");
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Writes the Java program correctly with import Scanner, public class, main() method, and proper syntax.	0.5
2	Takes the number of jerseys as input and checks if it is valid.	1
3	Takes the team choice as input and checks if it is valid.	1
4	Calculates the total price for the total number of jerseys correctly.	1
5	Extracts the first digit and the last two digits of the birth date correctly.	2
6	Calculates the lucky number using the correct formula.	2
7	Checks the discount condition and updates the bill correctly.	2
6	Displays the correct message and final amount to be paid.	0.5

SET - [B]

SOLUTION

```
import java.util.Scanner;

public class SetB {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of flags: ");
        int flag_num = sc.nextInt();

        if (flag_num > 0) {
            System.out.print("Enter team: 1-Brazil, 2-Portugal: ");
            int team = sc.nextInt();

            if (team == 1 || team == 2) {
                double bill = 55 * flag_num;

                System.out.println("Enter date of birth (DDMMYYYY): ");
                int birth_date = sc.nextInt();

                int last_digit = birth_date % 10;
                int first_two_digit = birth_date / 1000000;
                int lucky_number = 0;

                if (team == 1) {
                    lucky_number = (last_digit - 6) + (first_two_digit * 5);
                } else {
                    lucky_number = (last_digit - 7) + (first_two_digit * 8);
                }

                if (lucky_number % 4 == 0 && lucky_number % 8 != 0) {
                    bill = bill * 0.74;
                    System.out.println("Discount Applied");
                    System.out.println("Need to pay Tk. " + bill);
                } else {
                    System.out.println("No Discount Applied");
                    System.out.println("Need to pay Tk. " + bill);
                }
            } else {
                System.out.println("No team exists");
            }
        } else {
            System.out.println("Invalid number");
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Writes the Java program correctly with import Scanner, public class, main() method, and proper syntax.	0.5
2	Takes the number of flags as input and checks if it is valid.	1
3	Takes the team choice as input and checks if it is valid.	1
4	Calculates the total price for the total number of flags correctly.	1
5	Extracts the first two digits and the last digit of the birth date correctly.	2
6	Calculates the lucky number using the correct formula.	2
7	Checks the discount condition and updates the bill correctly.	2
8	Displays the correct message and final amount to be paid.	0.5

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

A space engineer is testing a new rocket by completing **N** launch stages. After completing the **n-th** stage, the rocket earns thrust points equal to **n!** (**n factorial**). If the thrust points are **divisible by both 2 and 3**, they are added to the **Main Fuel Reserve**. Otherwise, if they are **divisible by 2 but not by 3**, they are added to the **Backup Fuel Reserve**. All other thrust points are discarded. However, if the thrust points **exceed 100**, the engine reaches its safety limit. The engineer records the thrust points, prints **"Engine safety limit reached!"**, and immediately stops the test.

Write a Java program to simulate the rocket test. If the test stops because of the safety limit, print the thrust points at which it occurred and the warning message. Otherwise, after all launch stages are completed, print the total points stored in the **Main Fuel Reserve** and the **Backup Fuel Reserve**.

Factorial: $1! = 1$, $2! = 2 \times 1! = 2$, $3! = 3 \times 2! = 6$, $4! = 4 \times 3! = 24$

Sample Input	Sample Output
Enter number of launch stages: 4	Main Fuel Reserve = 30 Backup Fuel Reserve = 2 Explanation: $1! = 1 \rightarrow$ Discarded $2! = 2 \rightarrow$ Even but not divisible by 3 $\rightarrow$ Backup Fuel = 2 $3! = 6 \rightarrow$ Divisible by both 2 and 3 $\rightarrow$ Main Fuel = 6 $4! = 24 \rightarrow$ Divisible by both 2 and 3 $\rightarrow$ Main Fuel = 30
Enter number of launch stages: 8	Thrust Points: 120 Engine safety limit reached! Explanation: $1! = 1 \rightarrow$ Discarded $2! = 2 \rightarrow$ Backup Fuel $3! = 6 \rightarrow$ Main Fuel $4! = 24 \rightarrow$ Main Fuel $5! = 120 \rightarrow$ Thrust Points exceed 100 , test stops

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

A robotics engineer is testing a new assembly robot by completing **N** production stages. After completing the **n-th** stage, the robot produces energy units equal to **n!** (**n factorial**). If the energy units are **divisible by 4**, they are stored in the **Primary Battery**. Otherwise, if they are **even but not divisible by 4**, they are stored in the **Secondary Battery**. All other energy units are discarded. However, the robot has an overheating limit. If the energy units become **greater than 500**, the engineer records the energy units, prints "**Robot overheated!**", and immediately stops the test.

Write a Java program to simulate the robot test. If the test stops because of overheating, print the energy units at which it occurred and the warning message. Otherwise, after all production stages are completed, print the total energy stored in the **Primary Battery** and the **Secondary Battery**.

Factorial: $1! = 1$, $2! = 2 \times 1! = 2$, $3! = 3 \times 2! = 6$, $4! = 4 \times 3! = 24$

Sample Input	Sample Output
Enter number of production stages: 4	Primary Battery = 24 Secondary Battery = 8 Explanation: 1! = 1 → Discarded 2! = 2 → Even but not divisible by 4 → Secondary Battery = 2 3! = 6 → Even but not divisible by 4 → Secondary Battery = 8 4! = 24 → Divisible by 4 → Primary Battery = 24
Enter number of production stages: 7	Energy Units: 720 Robot overheated! Explanation: 1! = 1 → Discarded 2! = 2 → Secondary Battery 3! = 6 → Secondary Battery 4! = 24 → Primary Battery 5! = 120 → Primary Battery 6! = 720 → Energy units exceed 500, so the test stops immediately.

SET - [A]**SOLUTION**

```

import java.util.Scanner;
public class Exam2SetA{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number of launch stages: ");
        int n = sc.nextInt();
        int trustPoints = 1;
        int mainFuel = 0;
        int backupFuel = 0;
        boolean limitReached = false;
        for (int i = 1; i <= n; i++) {
            if (i > 1) {
                trustPoints *= i;
            }
            if (trustPoints % 2 == 0 && trustPoints % 3 == 0) {
                mainFuel += trustPoints;
            }
            else if (trustPoints % 2 == 0 && trustPoints % 3 != 0) {
                backupFuel += trustPoints;
            }
            if (trustPoints > 100) {
                System.out.println("Thrust Points: " + trustPoints);
                System.out.println("Engine safety limit reached!");
                limitReached = true;
                break;
            }
        }
        if (limitReached == false) {
            System.out.println("Main Fuel Reserve = " + mainFuel);
            System.out.println("Backup Fuel Reserve = " + backupFuel);
        }
    }
}

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Correct input, variable declarations, and initialization	1
2	Correct use of a loop to calculate thrust points using factorial incrementally	3
3	Correct implementation of the given conditional logic for the main fuel and backup fuel	2
4	Correct implementation of the stopping condition using break	2
5	Correct output (warning message or final totals)	2

SET - [B]**SOLUTION**

```

import java.util.Scanner;
public class Exam2SetB{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number of production stages: ");
        int n = sc.nextInt();
        int energyUnits = 1;
        int primaryBattery = 0;
        int secondaryBattery = 0;
        boolean overheated = false;
        for (int i = 1; i <= n; i++) {
            if (i > 1) {
                energyUnits *= i;
            }
            if (energyUnits % 4 == 0) {
                primaryBattery += energyUnits;
            }
            else if (energyUnits % 2 == 0) {
                secondaryBattery += energyUnits;
            }
            if (energyUnits > 500) {
                System.out.println("Energy Units: " + energyUnits);
                System.out.println("Robot overheated!");
                overheated = true;
                break;
            }
        }
        if (overheated == false) {
            System.out.println("Primary Battery = " + primaryBattery);
            System.out.println("Secondary Battery = " + secondaryBattery);
        }
    }
}

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Correct input, variable declarations, and initialization	1
2	Correct use of a loop to calculate energy units using factorial incrementally	3
3	Correct implementation of the given conditional logic for the primary battery and secondary battery	2
4	Correct implementation of the stopping condition using break	2
5	Correct output (warning message or final totals)	2

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

A hospital has multiple emergency wards. The first ward contains **1** patient bed, the second ward contains **2** patient beds, the third ward contains **3** patient beds, and so on. A nurse visits each bed to record the patient's body temperature, inspecting the wards one by one. A temperature **below 30°C or above 45°C** is considered invalid. In such cases, the nurse immediately takes the patient's temperature again. Invalid readings should not be included in any calculation. If a hospital-wide emergency occurs during the inspection, the nurse enters **-1 as a special input instead of a temperature**. Once **-1** is entered, the inspection is terminated immediately, and no further temperatures are recorded.

Write a Java program that **first takes the number of wards as input** and then records the temperature of every patient according to the rules above. After all inspections, display the **ward number with the highest number of patients with fever (temperature 38°C or above)**. If multiple wards have the same highest number of fever patients, display the **first such ward**. If the inspection is terminated due to an emergency, display the **ward number and bed number** where the emergency occurred.

Sample Input 1	Sample Input 2	Sample Input 3
Enter the number of wards: 3 Enter temperature(s) for :- Ward 1: 37 Ward 2: Bed 1: 40 Bed 2: 38 Ward 3: Bed 1: 38 Bed 2: 40 Bed 3: 37	Enter the number of wards: 3 Enter temperature(s) for :- Ward 1: Bed 1: 37 Ward 2: Bed 1: 46 Bed 1: 39 Bed 2: 36 Ward 3: Bed 1: 38 Bed 2: 40 Bed 3: 41	Enter the number of wards: 4 Enter temperature(s) for :- Ward 1: Bed 1: 36 Ward 2: Bed 1: 38 Bed 2: 37 Ward 3: Bed 1: 41 Bed 2: -1
Sample Output 1	Sample Output 2	Sample Output 3
Ward 2 has the highest number of patients with fever.	Invalid temperature! Enter temperature again for Bed 1 at Ward 2 Ward 3 has the highest number of patients with fever.	Emergency occurred at Ward 3, Bed 2 Ward 2 has the highest number of patients with fever.

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

An airport has multiple security checkpoints. The first checkpoint screens **1** passenger, the second checkpoint screens **2** passengers, the third checkpoint screens **3** passengers, and so on. A security officer checks each passenger one by one to record the weight of their carry-on baggage. A baggage weight **below 0 kg or above 15 kg** is considered an invalid reading. In such cases, the officer immediately measures the baggage again. Invalid readings should not be included in any calculation. If an airport-wide emergency occurs during the screening process, the officer enters **-1** as a special input instead of a baggage weight. Once **-1** is entered, the screening process is terminated immediately, and no further passengers are screened.

Write a Java program that first takes the number of security checkpoints as input and then records the baggage weight of every passenger according to the rules above. After all passengers have been screened, display the **checkpoint number with the highest number of passengers carrying heavy baggage** (baggage weight **10 kg or above**). If multiple checkpoints have the same highest number of heavy-baggage passengers, display the **first such checkpoint**. If the screening process is terminated due to an emergency, display the **checkpoint number and passenger number** where the emergency occurred.

Sample Input 1	Sample Input 2	Sample Input 3
Enter the number of checkpoints: 3 Enter weight(s) for:- Checkpoint 1: Passenger 1: 8 Checkpoint 2: Passenger 1: 10 Passenger 2: 12 Checkpoint 3: Passenger 1: 9 Passenger 2: 11 Passenger 3: 10	Enter the number of checkpoints: 3 Enter weight(s) for:- Checkpoint 1: Passenger 1: 7 Checkpoint 2: Passenger 1: 18 Passenger 1: 11 Passenger 2: 9 Checkpoint 3: Passenger 1: 10 Passenger 2: 12 Passenger 3: 13	Enter the number of checkpoints: 4 Enter weight(s) for:- Checkpoint 1: Passenger 1: 8 Checkpoint 2: Passenger 1: 10 Passenger 2: 9 Checkpoint 3: Passenger 1: 11 Passenger 2: -1
Sample Output 1	Sample Output 2	Sample Output 3
Checkpoint 2 has the highest number of passengers with heavy baggage.	Invalid baggage weight! Enter weight again for Passenger 1 at Checkpoint 2 Checkpoint 3 has the highest number of passengers with heavy baggage.	Emergency occurred at Checkpoint 3, Passenger 2 Checkpoint 2 has the highest number of passengers with heavy baggage.

SET - [A]

SOLUTION

```

import java.util.Scanner;
public class Exam3SetA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of wards: ");
        int wards = sc.nextInt();
        int maxFever = -1;
        int wardWithMaxFever = 1;
        boolean emergency = false;
        System.out.println("Enter temperature(s) for:- ");
        for (int ward = 1; ward <= wards; ward++) {
            System.out.println("Ward " + ward + ":");
            int feverCount = 0;
            for (int bed = 1; bed <= ward; bed++) {
                System.out.print("Bed " + bed + ": ");
                int temp = sc.nextInt();
                if (temp == -1) {
                    emergency = true;
                    System.out.println("Emergency occurred at Ward " + ward + ", Bed " + bed);
                    break;
                }
                if (temp < 30 || temp > 45) {
                    System.out.println("Invalid temperature! Enter temperature again for Bed "
                        + bed + " at Ward " + ward);
                    bed -= 1;
                    continue;
                }
                if (temp >= 38) {
                    feverCount++;
                }
            }
            if (emergency==true) {
                break;
            }
            else if ((emergency==false) && (feverCount > maxFever)) {
                maxFever = feverCount;
                wardWithMaxFever = ward;
            }
        }
        System.out.println("Ward "+wardWithMaxFever+" has the highest number of patients with fever.");
    }
}

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Correct use of nested loops (outer loop for wards and inner loop for beds)	2.5
2	Correctly asks for the temperature again if an invalid temperature (below 30°C or above 45°C) is entered	2
3	Correctly counts the number of fever patients (temperature 38°C or above) in each ward	1.5
4	Correctly identifies the ward with the highest number of fever patients (displays the first ward in case of a tie)	1.5
5	Correctly stops the inspection when -1 is entered	1.5
6	Correctly displays the ward number and bed number where the emergency occurred	1

SET - [B]

SOLUTION

```
import java.util.Scanner;
public class Exam3SetB {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of checkpoints: ");
        int checkpoints = sc.nextInt();
        int maxHeavy = -1;
        int checkpointWithMaxHeavy = 1;
        boolean emergency = false;
        System.out.println("Enter weight(s) for:- ");
        for (int checkpoint = 1; checkpoint <= checkpoints; checkpoint++) {
            System.out.println("Checkpoint " + checkpoint + ":");
            int heavyCount = 0;
            for (int passenger = 1; passenger <= checkpoint; passenger++) {
                System.out.print("Passenger " + passenger + ": ");
                int weight = sc.nextInt();
                if (weight == -1) {
                    emergency = true;
                    System.out.println("Emergency occurred at Checkpoint " + checkpoint + ",
Passenger " + passenger);
                    break;
                }
                if (weight < 0 || weight > 15) {
                    System.out.println("Invalid baggage weight! Enter weight again for
Passenger " + passenger + " at Checkpoint " + checkpoint);
                    passenger -= 1;
                    continue;
                }
                if (weight >= 10) {
                    heavyCount++;
                }
            }
            if (emergency==true) {
                break;
            }
            else if ((emergency==false) && (heavyCount > maxHeavy)) {
                maxHeavy = heavyCount;
                checkpointWithMaxHeavy = checkpoint;
            }
        }
        System.out.println("Checkpoint "+checkpointWithMaxHeavy+" has the highest number of passengers
with heavy baggage.");
    }
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Correct use of nested loops (outer loop for checkpoints and inner loop for passengers)	2.5
2	Correctly asks for the baggage weight again if an invalid weight (below 0kg or above 15kg) is entered	2
3	Correctly counts the number of passengers with the heaviest baggage (weight 10kg or above) in each checkpoint	1.5
4	Correctly identifies the checkpoint with the highest number of passengers with the heaviest baggage (displays the first ward in case of a tie)	1.5
5	Correctly stops the inspection when -1 is entered	1.5
6	Correctly displays the checkpoint number and passenger number where the emergency occurred	1

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

BRAC University wants to encrypt employee codes before storing them in its system. Write a Java program that takes an employee code as input and generates an encrypted version of the code by processing each character according to the following rules:

1. **Lowercase letters (a - z)**
 - a. Shift each **even-index lowercase letter forward by 2 positions**; **odd-index lowercase letters remain unchanged**.
 - b. If the shift goes beyond 'z', wrap around and continue again from 'a'.
2. **Uppercase letters (A - Z)**
 - a. Shift each **odd-index uppercase letter backward by 1 position**; **even-index uppercase letters will remain unchanged**.
 - b. If the shift goes before 'A', wrap around and continue again from 'Z'.
3. **Digits (0 - 9)**
 - a. Replace every **even digit** with *.
 - b. Replace every **odd digit** with #.
4. **Special characters** (such as @, #, \$, %, !, etc): Replace every special character with '-'.

Finally, **print** the encrypted employee code.

Sample Input	Sample Output	Explanation
Enter Employee Code: zyAZ38@!	Encrypted Code: byAY#*--	z → lowercase + even index → + 2 → b y → lowercase + odd index → unchanged → y A → uppercase + even index → unchanged → A Z → uppercase + odd index → -1 → Y 3 → odd digit → # 8 → even digit → * @ → special character → - ! → special character → - Output : byAY#*--

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

Foodpanda wants to encrypt employee codes before storing them in its system. Write a Java program that takes an employee code as input and generates an encrypted version of the code by processing each character according to the following rules:

1. Lowercase letters (a - z)

- Shift each **odd-index lowercase letter backward by 2 positions**; even-index lowercase letters remain unchanged.
- If the shift goes before 'a', wrap around and continue again from 'z'.

2. Uppercase letters (A - Z)

- Shift each **even-index uppercase letter forward by 1 position**; odd-index uppercase letters remain unchanged.
- If the shift goes beyond 'Z', wrap around and continue again from 'A'.

3. Digits (0 - 9) :

- Replace every **even digit** with #.
- Replace every **odd digit** with *.

4. Special characters (such as @, #, \$, %, !): Replace every special character with '='.

Finally, **print** the encrypted employee code.

Sample Input	Sample Output	Explanation
Enter Employee Code: baZY38@!	Encrypted Code: byAY*#==	b → lowercase + even index → unchanged → b a → lowercase + odd index → -2 → y Z → uppercase + even index → +1 → A Y → uppercase + odd index → unchanged → Y 3 → odd digit → * 8 → even digit → # @ → special character → = ! → special character → = Output : byAY*#==

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class CodeEncryption1 {
    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter Employee Code : ");
        String code = scanner.nextLine();

        String encryptedCode = "";

        for (int i = 0; i < code.length(); i++) {

            char ch = code.charAt(i);

            // Lowercase letters
            if (ch >= 'a' && ch <= 'z') {

                if (i % 2 == 0) {
                    ch = (char) (ch + 2);

                    if (ch > 'z') {
                        ch = (char) (ch - 26);
                    }
                }
            }

            // Uppercase letters
            else if (ch >= 'A' && ch <= 'Z') {

                if (i % 2 == 1) {
                    ch = (char) (ch - 1);

                    if (ch < 'A') {
                        ch = 'Z';
                    }
                }
            }

            // Digits
            else if (ch >= '0' && ch <= '9') {

                int digit = ch - '0';

                if (digit % 2 == 0) {
                    ch = '*';
                } else {
                    ch = '#';
                }
            }
        }
    }
}
```

```

        // Special characters
        else {
            ch = '-';
        }

        encryptedCode = encryptedCode + ch;
    }

    System.out.println("Encrypted Code : " + encryptedCode);
}
}

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Takes input and processes each character using a loop	2
2	Correctly handles lowercase letters based on index, including wrap-around	2
3	Correctly handles uppercase letters based on index, including wrap-around	2
4	Correctly handles even/odd digits	2
5	Correctly handles special characters and prints the encrypted code	2

SOLUTION

```
import java.util.Scanner;

public class CodeEncryption2 {
    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter Employee Code : ");
        String code = scanner.nextLine();

        String encryptedCode = "";

        for (int i = 0; i < code.length(); i++) {

            char ch = code.charAt(i);

            // Lowercase letters
            if (ch >= 'a' && ch <= 'z') {

                if (i % 2 == 1) {
                    ch = (char) (ch - 2);

                    if (ch < 'a') {
                        ch = (char) (ch + 26);
                    }
                }
            }

            // Uppercase letters
            else if (ch >= 'A' && ch <= 'Z') {

                if (i % 2 == 0) {
                    ch = (char) (ch + 1);

                    if (ch > 'Z') {
                        ch = 'A';
                    }
                }
            }

            // Digits
            else if (ch >= '0' && ch <= '9') {

                int digit = ch - '0';

                if (digit % 2 == 0) {
                    ch = '#';
                }
                else {
                    ch = '*';
                }
            }
        }
    }
}
```

```

        // Special characters
        else {
            ch = '=';
        }

        encryptedCode = encryptedCode + ch;
    }

    System.out.println("Encrypted Code : " + encryptedCode);
}
}

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Takes input and processes each character using a loop	2
2	Correctly handles lowercase letters based on index, including wrap-around	2
3	Correctly handles uppercase letters based on index, including wrap-around	2
4	Correctly handles even/odd digits	2
5	Correctly handles special characters and prints the encrypted code	2